EF Core 8.0 Hands-On Exercise – Retail Inventory System

Lab-01

Step 1: Create a New Console Application

A new .NET Console Application named **RetailInventory** was created using Visual Studio 2022.

Step 2: Install Required NuGet Packages

Open **Tools → NuGet Package Manager → Package Manager Console** and execute the following commands:

```
Install-Package Microsoft.EntityFrameworkCore -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.Design -Version 8.0.8
```

Purpose of Packages

Package	Purpose
Microsoft.EntityFrameworkCore	Provides the core Entity Framework functionality.
Microsoft.EntityFrameworkCore.SqlServer	Enables EF Core to connect with SQL Server.
Microsoft.EntityFrameworkCore.Design	Supports design-time features such as migrations and scaffolding.

Step 3: Project Structure

```
RetailInventory
|
├── Models
|   ├── Category.cs
|   └── Product.cs
|
├── AppDbContext.cs
├── Program.cs
└── RetailInventory.csproj
```

Step 4: Create Model Classes

Category.cs

Represents the **Category** table in the database.

```
public class Category
{
    public int Id { get; set; }
    public string Name { get; set; } = "";

    public List<Product> Products { get; set; } = new();
}
```

Explanation

- Id – Primary Key
- Name – Stores the category name.
- Products – Navigation property representing the one-to-many relationship between Category and Product.

Product.cs

Represents the **Product** table in the database.

```
public class Product
{
   public int Id { get; set; }
   public string Name { get; set; } = "";
   public decimal Price { get; set; }
   public int StockQuantity { get; set; }

   public int CategoryId { get; set; }
   public Category Category { get; set; } = null!;
}
```

Explanation

- Id – Primary Key
- Name – Product Name
- Price – Product Price
- StockQuantity – Available stock
- CategoryId – Foreign Key
- Category – Navigation property to access category details.

Step 5: Create Database Context

Create **AppDbContext.cs**

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;

public class AppDbContext : DbContext
{
   public DbSet<Product> Products { get; set; }
   public DbSet<Category> Categories { get; set; }

   protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
   {
      optionsBuilder.UseSqlServer(
         @"Server=(localdb)\MSSQLLocalDB;Database=RetailInventoryDb;Trusted_Connection=True;"
      );
```

```
    }
}
```

Explanation

- DbContext acts as a bridge between the application and SQL Server.
- DbSet<Product> maps the Product class to the Product table.
- DbSet<Category> maps the Category class to the Category table.
- UseSqlServer() configures the SQL Server database connection.

Step 6: Connection String

```
@"Server=(localdb)\MSSQLLocalDB;
Database=RetailInventoryDb;
Trusted_Connection=True;"
```

Connection Details

Property	Description
Server	SQL Server LocalDB Instance
Database	RetailInventoryDb
Trusted_Connection=True	Uses Windows Authentication

Step 7: Program.cs

```
using Microsoft.EntityFrameworkCore;
using RetailInventory;
using RetailInventory.Models;

using var context = new AppDbContext();

context.Database.EnsureCreated();

if (!context.Categories.Any())
{
    var electronics = new Category
    {
        Name = "Electronics"
    };

    var product = new Product
    {
        Name = "Laptop",
        Price = 55000,
        StockQuantity = 10,
        Category = electronics
    };
```

```
    context.Products.Add(product);
    context.SaveChanges();
}

var products = context.Products
                .Include(p => p.Category)
                .ToList();

foreach (var p in products)
{
    Console.WriteLine($"Product: {p.Name}, Category: {p.Category.Name}, Price: {p.Price}, Stock:
{p.StockQuantity}");
}
```

Step 8: How the Code Works

Create Database

```
context.Database.EnsureCreated();
```

Creates the database automatically if it does not already exist.

Create Category

```
var electronics = new Category
{
    Name = "Electronics"
};
```

Creates a new category object.

Create Product

```
var product = new Product
{
    Name = "Laptop",
    Price = 55000,
    StockQuantity = 10,
    Category = electronics
};
```

Creates a new product and associates it with the Electronics category.

Add Data

```
context.Products.Add(product);
```

Adds the product object to the Entity Framework context.

Save Data

```
context.SaveChanges();
```

Saves all changes to the SQL Server database.

Retrieve Data

```
context.Products
    .Include(p => p.Category)
    .ToList();
```

Retrieves all products along with their related category details.

Display Output

```
Console.WriteLine($"Product: {p.Name}, Category: {p.Category.Name}, Price: {p.Price}, Stock: {p.StockQuantity}");
```

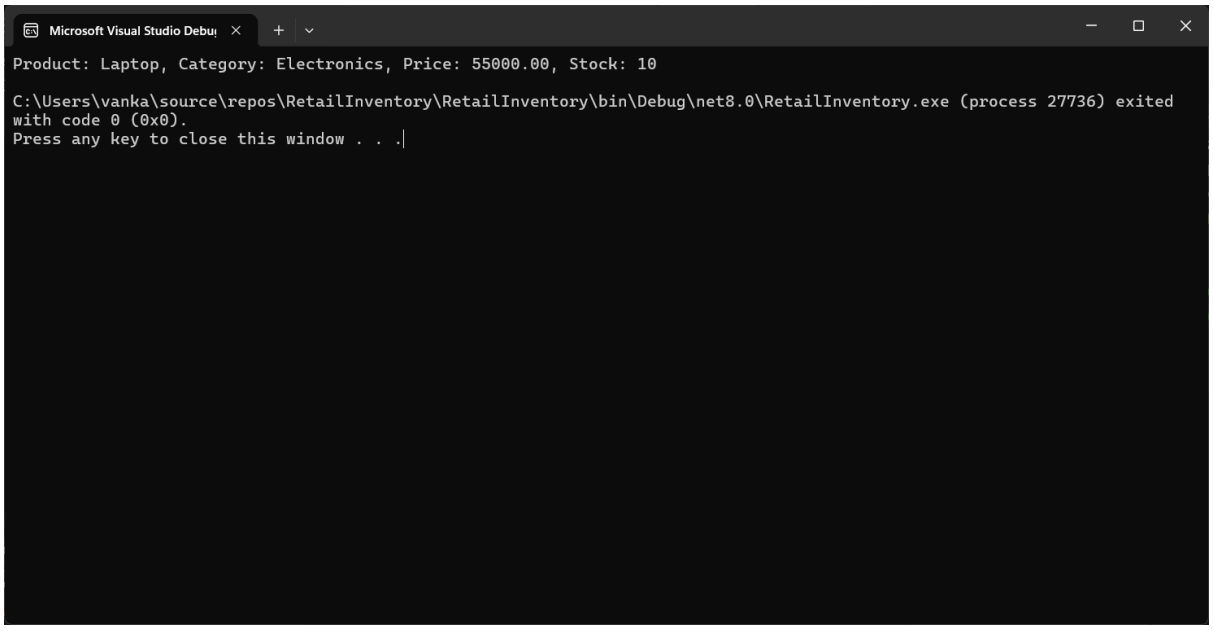
Displays the product information in the console.

How Everything is Connected

Category.cs
↓
Product.cs
↓
AppDbContext.cs
↓
SQL Server LocalDB
↓
Program.cs
↓
Console Output

Output

Product: Laptop, Category: Electronics, Price: 55000.00, Stock: 10



Lab 2: Setting Up the Database Context for a Retail Store

Scenario

The retail store needs to store product and category information in a SQL Server database. Entity Framework Core (EF Core) is used to establish a connection between the C# application and SQL Server using a database context (DbContext).

Objective

The objective of this lab is to configure the **DbContext**, establish a connection to **SQL Server LocalDB**, and create model classes for storing product and category information.

Software Requirements

- Visual Studio 2022
- .NET 8.0 SDK
- SQL Server LocalDB
- Entity Framework Core 8.0
- NuGet Package Manager

Project Name

RetailInventory

Step 1: Create a New Console Application

A new .NET Console Application named **RetailInventory** was created using Visual Studio 2022.

Step 2: Install Required NuGet Packages

Open:

Tools → NuGet Package Manager → Package Manager Console

Execute the following commands:

```
Install-Package Microsoft.EntityFrameworkCore -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.Design -Version 8.0.8
```

Purpose of Installed Packages

Package	Purpose
Microsoft.EntityFrameworkCore	Provides the core functionality of Entity Framework Core.
Microsoft.EntityFrameworkCore.SqlServer	Enables the application to connect to SQL Server.

Package	Purpose
Microsoft.EntityFrameworkCore.Design	Supports design-time features such as migrations and scaffolding.

Step 3: Project Structure

```
RetailInventory
|
├── Models
|   ├── Category.cs
|   └── Product.cs
|
├── AppDbContext.cs
├── Program.cs
└── RetailInventory.csproj
```

Step 4: Create Model Classes

Category.cs

```
namespace RetailInventory.Models
{
   public class Category
   {
      public int Id { get; set; }

      public string Name { get; set; } = "";

      public List<Product> Products { get; set; } = new();
   }
}
```

Explanation

- Id – Primary Key of the Category table.
- Name – Stores the category name.
- Products – Navigation property representing the one-to-many relationship between Category and Product.

Product.cs

```
namespace RetailInventory.Models
{
   public class Product
   {
      public int Id { get; set; }

      public string Name { get; set; } = "";
```

```
    public decimal Price { get; set; }

    public int StockQuantity { get; set; }

    public int CategoryId { get; set; }

    public Category Category { get; set; } = null!;
  }
}
```

Explanation

- Id – Primary Key of the Product table.
- Name – Stores the product name.
- Price – Stores the product price.
- StockQuantity – Stores the available stock.
- CategoryId – Foreign Key that links the product to the Category table.
- Category – Navigation property used to access category information.

Step 5: Create AppDbContext

A class named **AppDbContext** was created by inheriting from the **DbContext** class. It acts as the bridge between the C# application and SQL Server.

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;

namespace RetailInventory
{
  public class AppDbContext : DbContext
  {
    public DbSet<Product> Products { get; set; }

    public DbSet<Category> Categories { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
      optionsBuilder.UseSqlServer(
        @"Server=(localdb)\MSSQLLocalDB;
          Database=RetailInventoryDb;
          Trusted_Connection=True;");
    }
  }
}
```

Explanation

- DbContext manages the database connection.
- DbSet<Product> maps the Product class to the **Products** table.
- DbSet<Category> maps the Category class to the **Categories** table.
- UseSqlServer() configures the SQL Server connection using the connection string.

Step 6: Configure SQL Server Connection

The SQL Server connection string was configured inside the OnConfiguring() method.

```
@"Server=(localdb)\MSSQLLocalDB;
Database=RetailInventoryDb;
Trusted_Connection=True;"
```

Connection Details

Property	Description
Server	Connects to SQL Server LocalDB
Database	Creates or connects to the RetailInventoryDb database
Trusted_Connection=True	Uses Windows Authentication

Step 7: Test the Database Connection

The Program.cs file was updated to create the database, insert sample data, and retrieve the stored records.

```
using Microsoft.EntityFrameworkCore;
using RetailInventory;
using RetailInventory.Models;

using var context = new AppDbContext();

context.Database.EnsureCreated();

if (!context.Categories.Any())
{
   var category = new Category
   {
      Name = "Electronics"
   };

   var product = new Product
   {
      Name = "Laptop",
      Price = 55000,
      StockQuantity = 10,
      Category = category
   };
```

```
    context.Products.Add(product);
    context.SaveChanges();
}

var products = context.Products
                .Include(p => p.Category)
                .ToList();

foreach (var p in products)
{
    Console.WriteLine($"Product: {p.Name}, Category: {p.Category.Name}, Price: {p.Price}, Stock:
{p.StockQuantity}");
}
```

Explanation

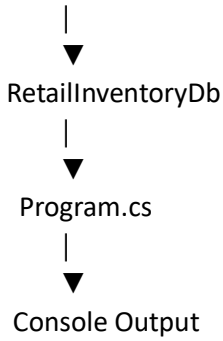
- EnsureCreated() creates the database if it does not exist.
- Add() adds a new product to the EF Core context.
- SaveChanges() stores the data in SQL Server.
- Include() retrieves the related Category data.
- ToList() fetches all product records.
- Console.WriteLine() displays the retrieved data.

Step 8: Build and Run the Application

1. Build the project using **Build → Rebuild Solution**.
2. Run the application using **Ctrl + F5**.
3. EF Core connects to SQL Server LocalDB.
4. The RetailInventoryDb database is created (if it does not already exist).
5. Product and Category data are inserted into the database.
6. The inserted records are retrieved and displayed in the console.

How the Application Connects to SQL Server

```
Category.cs + Product.cs
        |
        ▼
    AppDbContext.cs
        |
        ▼
  UseSqlServer(Connection String)
        |
        ▼
   SQL Server LocalDB
```

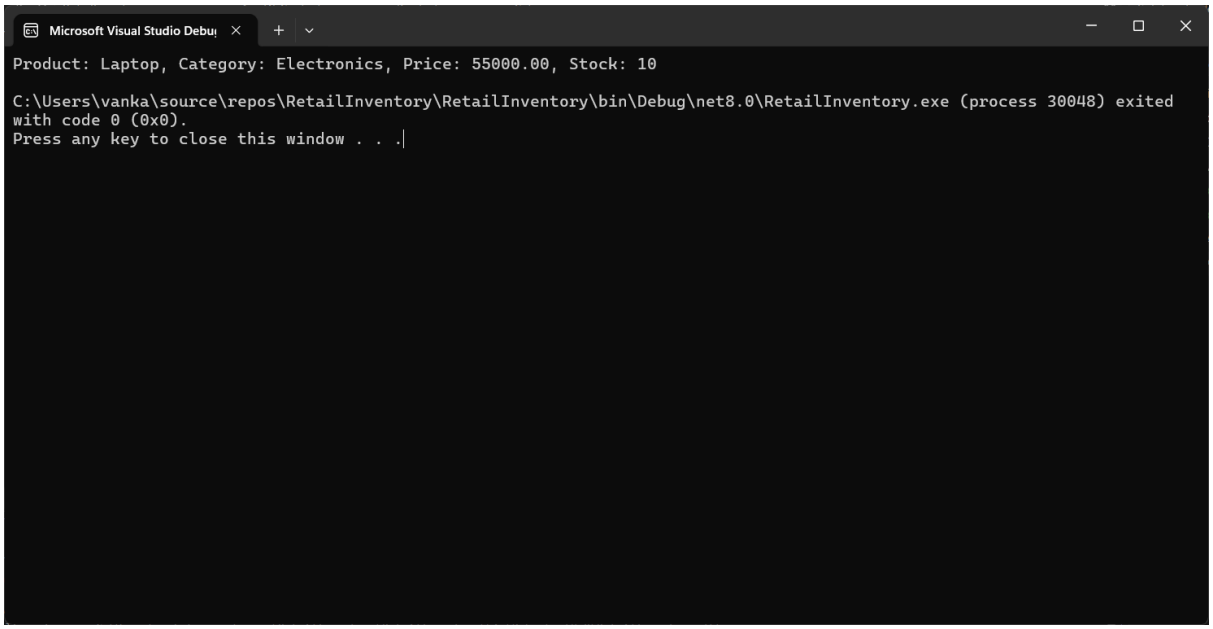


Output

Product: Laptop, Category: Electronics, Price: 55000.00, Stock: 10

Result

The **AppDbContext** was successfully configured to connect the application with **SQL Server LocalDB** using Entity Framework Core. The Product and Category classes were mapped to database tables through DbSet, and the application successfully created the database, inserted sample data, retrieved the records using LINQ, and displayed the output in the console.



Lab 3: Using EF Core CLI to Create and Apply Migrations

Scenario

The retail store database needs to be created based on the model classes defined in the project. EF Core migrations are used to generate and apply database schema changes.

Objective

The objective of this lab is to use EF Core migration commands to create the database and tables from the C# model classes.

Project Name

RetailInventory

Implementation

Step 1: Installed Required EF Core Packages

The following packages were installed in the project:

Install-Package Microsoft.EntityFrameworkCore -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.Design -Version 8.0.8
Install-Package Microsoft.EntityFrameworkCore.Tools -Version 8.0.8

Step 2: Category.cs

```
namespace RetailInventory.Models
{
   public class Category
   {
      public int Id { get; set; }

      public string Name { get; set; } = "";

      public List<Product> Products { get; set; } = new();
   }
}
```

Step 3: Product.cs

```
namespace RetailInventory.Models
{
   public class Product
   {
      public int Id { get; set; }

      public string Name { get; set; } = "";

      public decimal Price { get; set; }

      public int StockQuantity { get; set; }

      public int CategoryId { get; set; }

      public Category Category { get; set; } = null!;
   }
}
```

Step 4: AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;
```

```
namespace RetailInventory
{
   public class AppDbContext : DbContext
   {
      public DbSet<Product> Products { get; set; }

      public DbSet<Category> Categories { get; set; }

      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
      {
         optionsBuilder.UseSqlServer(

@"Server=(localdb)\MSSQLLocalDB;Database=RetailInventoryDb;Trusted_Connection=True;"
         );
      }
   }
}
```

Step 5: Program.cs

```
using Microsoft.EntityFrameworkCore;
using RetailInventory;
using RetailInventory.Models;

using var context = new AppDbContext();

if (!context.Categories.Any())
{
   var category = new Category
   {
      Name = "Electronics"
   };

   var product = new Product
   {
      Name = "Laptop",
      Price = 55000,
      StockQuantity = 10,
      Category = category
   };

   context.Products.Add(product);
   context.SaveChanges();
}

var products = context.Products
                  .Include(p => p.Category)
                  .ToList();
```

```
foreach (var p in products)
{
   Console.WriteLine($"Product: {p.Name}, Category: {p.Category.Name}, Price: {p.Price}, Stock: {p.StockQuantity}");
}
```

Step 6: Build the Project

The project was rebuilt using:

Build → Rebuild Solution

The build was successful.

Step 7: Create Initial Migration

In Visual Studio, the Package Manager Console was opened using:

Tools → NuGet Package Manager → Package Manager Console

The following command was executed:

Add-Migration InitialCreate

This command created a **Migrations** folder in the project. The migration file contains schema information for creating the **Categories** and **Products** tables.

Step 8: Apply Migration to Database

The following command was executed:

Update-Database

This command applied the migration and created the database schema in SQL Server LocalDB.

Step 9: Handling Existing Database Issue

Since the database was previously created using EnsureCreated(), the following error occurred:

There is already an object named 'Categories' in the database.

To fix this issue, the old database was removed using:

Drop-Database

After confirmation, the migration was applied again:

Update-Database

Step 10: Database Created

After applying the migration, EF Core created the database:

RetailInventoryDb

The following tables were created:

Categories
Products

Step 11: Run the Application

The application was executed using:

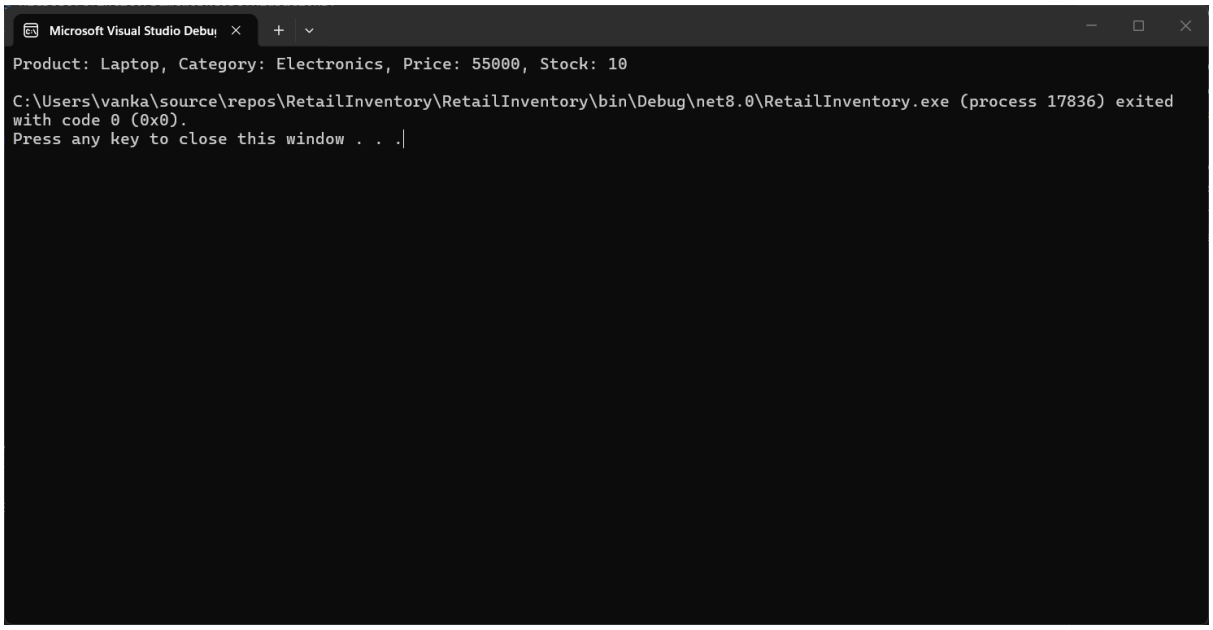
Ctrl + F5

Output

Product: Laptop, Category: Electronics, Price: 55000.00, Stock: 10

Result

EF Core migrations were successfully used to create and apply the database schema. The migration generated the required database structure based on the model classes. The database **RetailInventoryDb** was created in SQL Server LocalDB with **Products** and **Categories** tables. The application successfully inserted and retrieved data from the database.



Lab 4: Inserting Initial Data into the Database

Scenario

The retail store manager wants to add initial product categories and products to the system. Entity Framework Core is used to insert the initial records into the SQL Server database asynchronously.

Objective

To use EF Core asynchronous methods AddRangeAsync() and SaveChangesAsync() to insert initial data into the database.

Project Name

RetailInventory

Implementation

Step 1: Open the Existing Project

Continue with the existing **RetailInventory** project created in the previous labs.

Step 2: Verify Model Classes

Ensure the following model classes already exist:

- Category.cs
- Product.cs

These classes represent the **Categories** and **Products** tables in SQL Server.

Step 3: Verify Database Context

Ensure AppDbContext.cs is already configured with SQL Server LocalDB.

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;

namespace RetailInventory
{
   public class AppDbContext : DbContext
   {
      public DbSet<Product> Products { get; set; }

      public DbSet<Category> Categories { get; set; }

      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
      {
         optionsBuilder.UseSqlServer(

@"Server=(localdb)\MSSQLLocalDB;Database=RetailInventoryDb;Trusted_Connection=True;");
      }
   }
}
```

Step 4: Update Program.cs

Replace the contents of **Program.cs** with the following code.

```
using Microsoft.EntityFrameworkCore;
using RetailInventory;
using RetailInventory.Models;

using var context = new AppDbContext();

if (!await context.Categories.AnyAsync())
{
   var electronics = new Category
   {
      Name = "Electronics"
```

```
    };

    var groceries = new Category
    {
        Name = "Groceries"
    };

    await context.Categories.AddRangeAsync(electronics, groceries);

    var product1 = new Product
    {
        Name = "Laptop",
        Price = 75000,
        StockQuantity = 10,
        Category = electronics
    };

    var product2 = new Product
    {
        Name = "Rice Bag",
        Price = 1200,
        StockQuantity = 25,
        Category = groceries
    };

    await context.Products.AddRangeAsync(product1, product2);

    await context.SaveChangesAsync();

    Console.WriteLine("Initial data inserted successfully!");
}

var products = await context.Products
                    .Include(p => p.Category)
                    .ToListAsync();

foreach (var p in products)
{
    Console.WriteLine($"Product: {p.Name}, Category: {p.Category.Name}, Price: {p.Price}, Stock:
{p.StockQuantity}");
}
```

Step 5: Build the Project

Build the application by selecting:

Build → Rebuild Solution

Ensure the build completes successfully without errors.

Step 6: Run the Application

Execute the application using:

Ctrl + F5

The application connects to SQL Server, checks whether category records already exist, inserts the initial categories and products asynchronously, saves them into the database, and retrieves the inserted records.

Step 7: Verify the Database

Open **SQL Server Management Studio (SSMS)**.

Connect to:

(localdb)\MSSQLLocalDB

Open:

Databases
↓
RetailInventoryDb
↓
Tables

Verify that:

- Categories table contains:
    - Electronics
    - Groceries
- Products table contains:
    - Laptop
    - Rice Bag

Expected Output

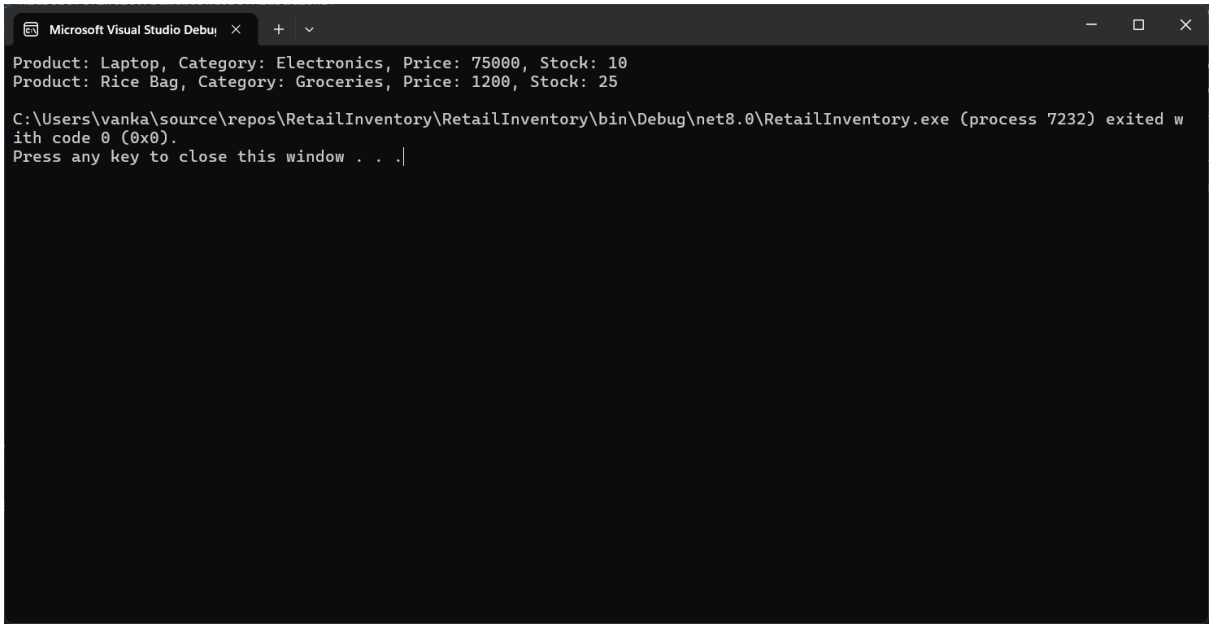
Initial data inserted successfully!

Product: Laptop, Category: Electronics, Price: 75000.00, Stock: 10

Product: Rice Bag, Category: Groceries, Price: 1200.00, Stock: 25

Result

The initial category and product records were successfully inserted into the SQL Server database using Entity Framework Core asynchronous methods AddRangeAsync() and SaveChangesAsync(). The data was successfully retrieved using LINQ and displayed in the console, demonstrating asynchronous database operations in EF Core.



Lab 5: Retrieving Data from the Database

Implementation

Step 1: Open the Existing Project

Continue using the existing **RetailInventory** project created in the previous labs.

Step 2: Verify Database

Ensure that the **RetailInventoryDb** database contains the product records inserted in **Lab 4**.

Step 3: Retrieve All Products

Open **Program.cs** and use the ToListAsync() method to retrieve all product records from the **Products** table.

```
var products = await context.Products.ToListAsync();

foreach (var p in products)
{
   Console.WriteLine($"{p.Name} - ₹{p.Price}");
}
```

Explanation:

- ToListAsync() retrieves all records from the **Products** table asynchronously.
- foreach is used to display each product's name and price.

Step 4: Retrieve a Product by ID

Use the FindAsync() method to retrieve a product using its primary key.

```
var product = await context.Products.FindAsync(1);

Console.WriteLine($"Found: {product?.Name}");
```

Explanation:

- FindAsync(1) searches for the product whose **Id = 1**.
- If the product exists, its name is displayed.

Step 5: Retrieve the First Product Matching a Condition

Use the FirstOrDefaultAsync() method to retrieve the first product whose price is greater than ₹50,000.

```
var expensive = await context.Products
                    .FirstOrDefaultAsync(p => p.Price > 50000);

Console.WriteLine($"Expensive: {expensive?.Name}");
```

Explanation:

- FirstOrDefaultAsync() returns the first record that satisfies the given condition.
- If no matching record exists, it returns null.

Step 6: Build the Project

Build the application using:

Build → Rebuild Solution

Ensure the project builds successfully.

Step 7: Run the Application

Run the application using:

Ctrl + F5

Expected Output

```
All Products:
Laptop - ₹75000.00
Rice Bag - ₹1200.00

Found: Laptop

Expensive: Laptop
```

Result

The application successfully retrieved data from the SQL Server database using Entity Framework Core asynchronous query methods. The ToListAsync() method was used to retrieve all products, FindAsync() was used to search by primary key, and FirstOrDefaultAsync() was used to retrieve the first record matching a specified condition.